Chapter 5 : Structural Design Patterns

Instructor: Abed Safadi, Ph.D.

References

- Richard N. Taylor, Nenad Medvidovie, Eric M. Dashofy: Software Architecture Foundations, Theory, and Practice.
- Marck Richards: Software Architecture Patterns.
- Erich Gamma, Richard Helm, Ralph Johnson & John Vlissides : Design Patterns : Elements of Reusable Object-Oriented Software.
- Robert C. Martin : Clean Architecture : A Craftsman's Guide to Software Structure and Design
- Websites notes

Structural patterns

- Structural patterns focus on the composition of objects and classes into larger structures while keeping these structures flexible and efficient.

- This composition refers to how these classes and objects are put together to achieve specific functionalities or behaviors.

- Structural patterns related to the problems of organization of objects in software. The essence of structural patterns is to organize the compositional structure of objects in a standard and efficient way.

Plan

- Flyweight
- Bridge
- Composite
- Proxy
- Adapter

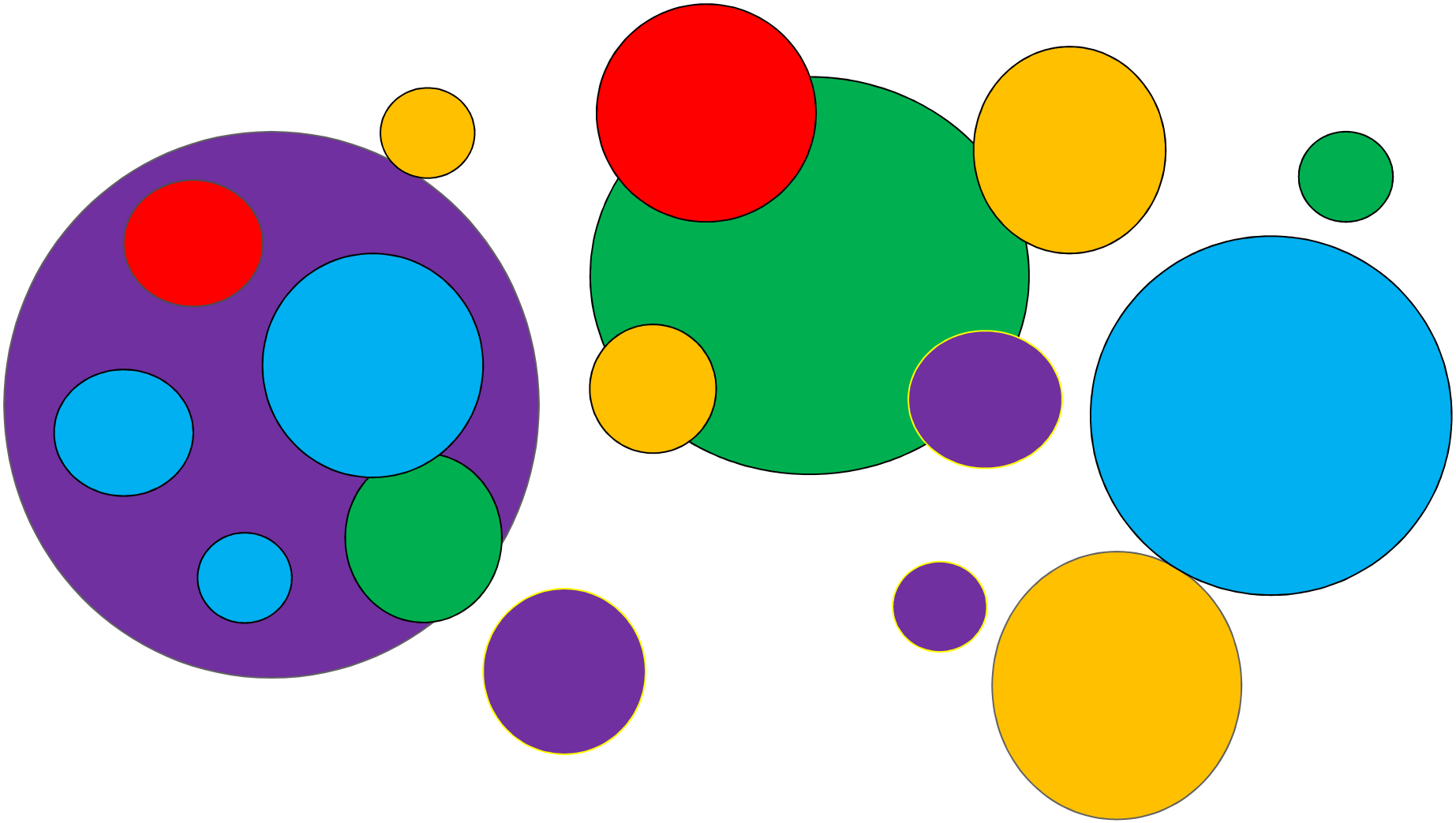
Flyweight pattern

Intent

- The "**Flyweight** pattern" is used to optimize the number of objects stored in your application's memory.
- **Flyweight** is a structural design pattern that allows you to efficiently utilize available memory by sharing common parts of state among multiple objects instead of keeping all of the data in each object.
- Used when you need to create a large number of similar objects
- To reduce memory usage you share Objects that are similar in some way rather than creating new ones

Example before Flyweight pattern

Creating 20 circles of different locations



Flyweight - Problem

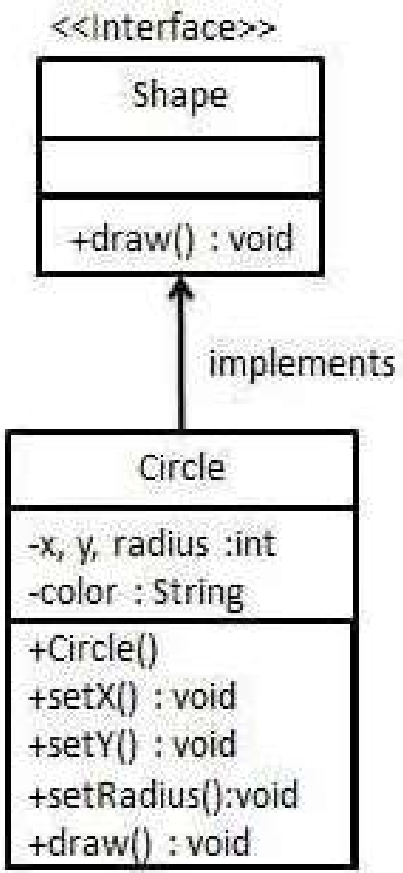
- Implementation : We are going to create a *Shape* interface and concrete class *Circle* implementing the *Shape* interface.

Shape.java

```
public interface Shape {
  void draw();
}
```

Circle.java

```
public class Circle implements Shape {
  private String color;
  private int x;
  private int y;
  private int radius;

  public Circle(String color){
    this.color = color;
  }

  public void setX(int x) {
    this.x = x;
  }
  public void setY(int y) {
    this.y = y;
   }
  public void setRadius(int radius) {
    this.radius = radius;
   }
  @Override
  public void draw() {
    System.out.println("Circle: Draw() [Color : " + color + ", x : " + x + ", y :" + y + ", radius :" + radius);
   }
}
```

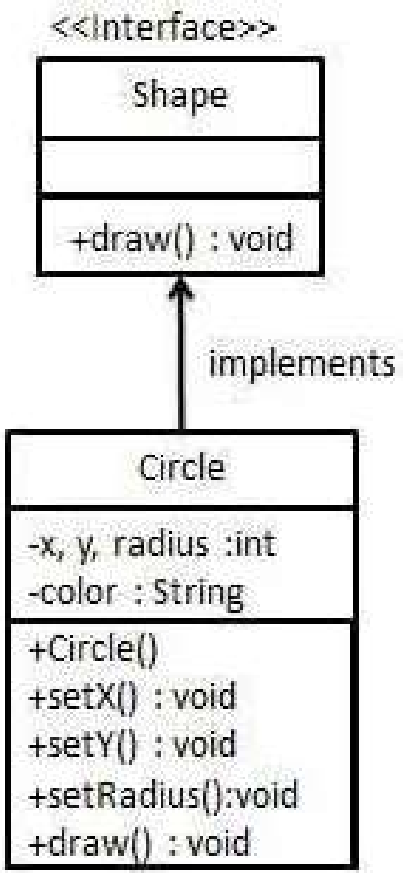


Flyweight - Problem

- Implementation : We are going to create a *Shape* interface and concrete class *Circle* implementing the *Shape* interface.

Demo.java

```
public class Demo {

  private static final String colors[] = { "Red", "Green", "Blue", “Black”,
    ”White”, };
  public static void main(String[] args) {
    for(int i=0; i < 20; ++i) {
      Circle circle = new Circle(color [(int)(Math.random()*colors.length)]);
        circle.setX(Math.random()*100);
        circle.setY(Math.random()*100);
        circle.setRadius(Math.random()*100);
        circle.draw();
    }
  }
}
```



Before Flyweight Design Patterns **we must create 20 objects** in order to draw 20 circles .

Flyweight - Solution

Flyweight pattern tries to reuse already existing similar kind objects by storing them and creates new object when no matching object is found.

A flyweight is an object through which we try to minimize memory usage by sharing data as much as possible.

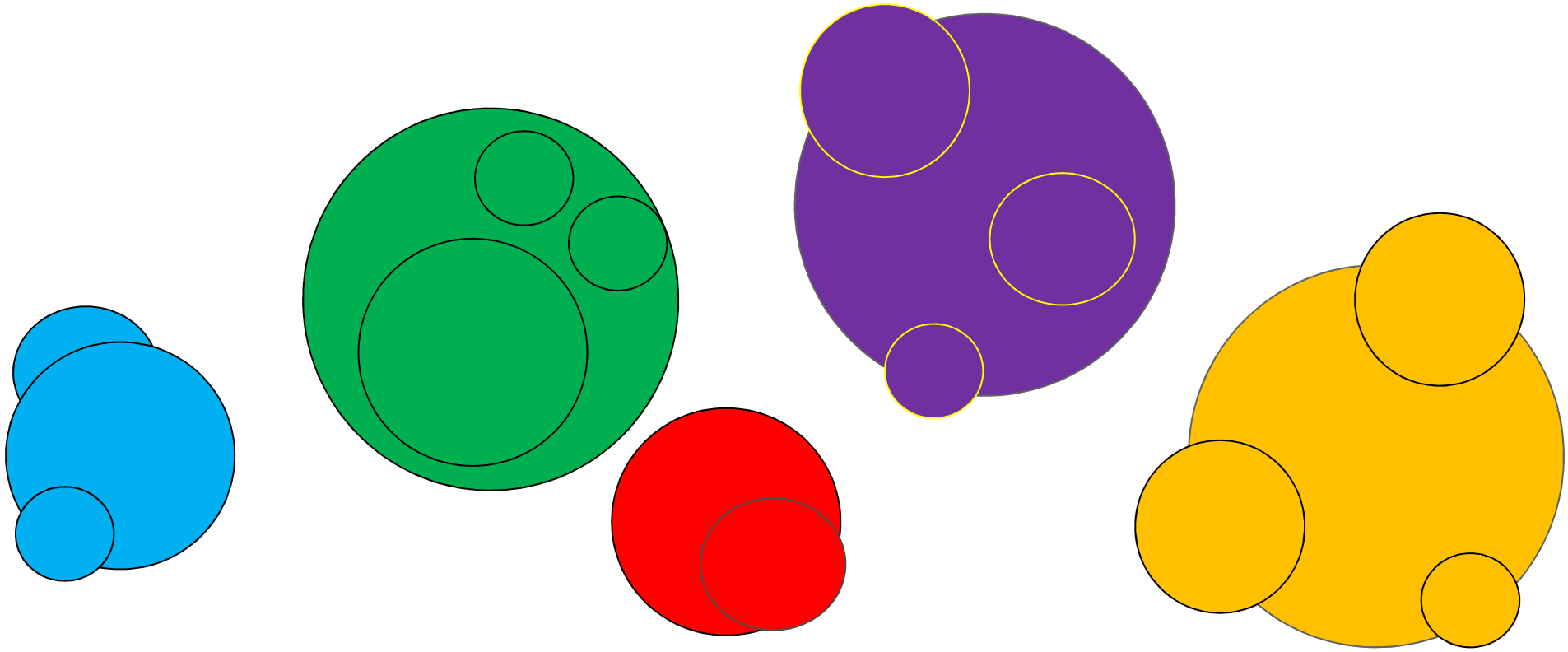
Two common terms are used here:
—*intrinsic* state : color
—*extrinsic* state: size.

The first category (intrinsic) can be stored in the flyweight and is shareable. The other one depends on the flyweight's context and is non-shareable.

➔We will demonstrate this pattern by drawing 20 circles of different locations but we will **create only 5 objects**

Flyweight - Solution

Only 5 colors are available so **color property (intrinsic state)** is used to **check** already existing *Circle* objects

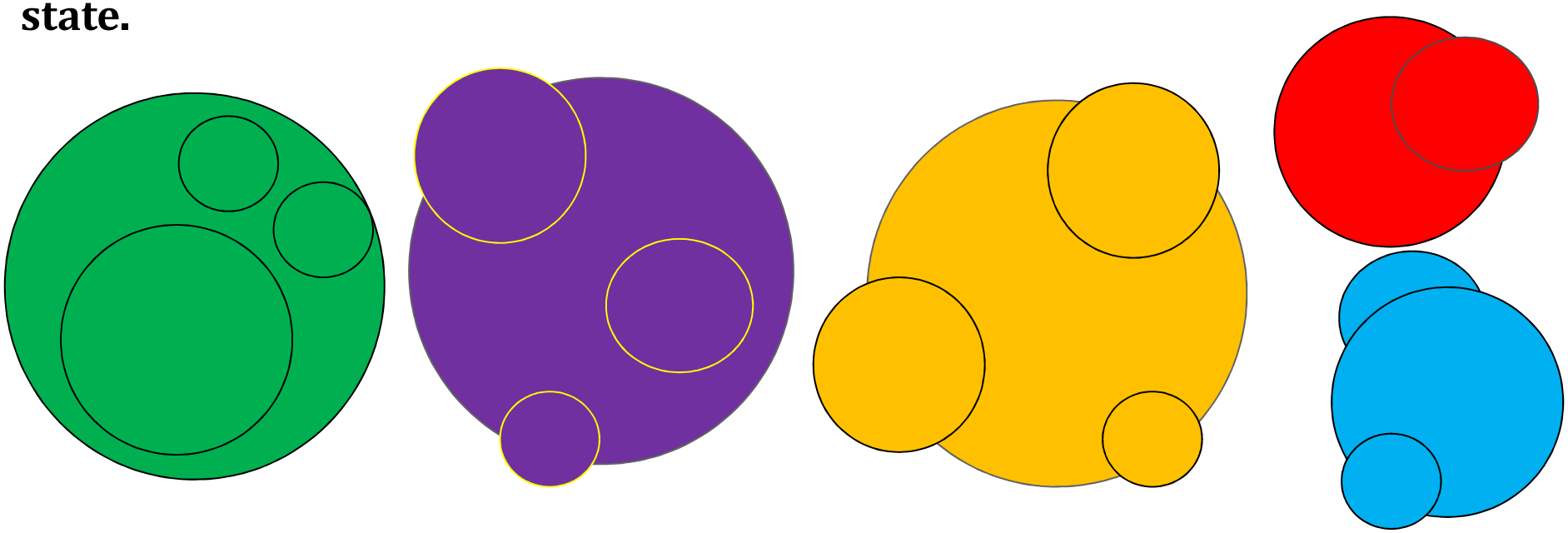


Flyweight - Solution

For example, in each group all circles have the same color. Other parts of a circle s state, such as radius is unique to each circle. After all, the values of this field change over time.

This data represents the always changing context in which the circle exists, while the color remain constant for each circle.

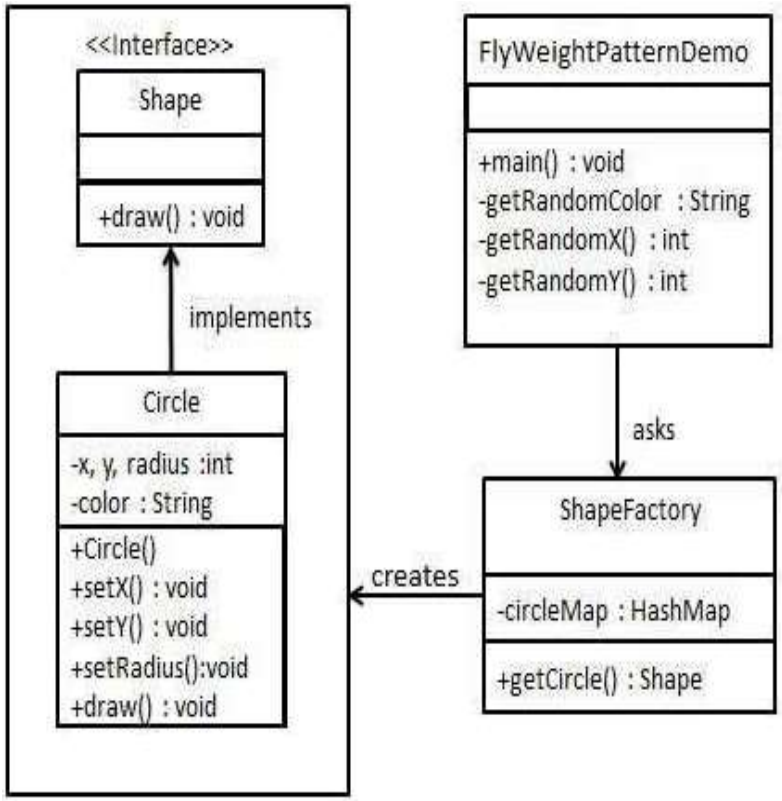
This constant data of an object is usually called the ***intrinsic state*****.** It lives within the object; other objects can only read it, not change it. The rest of the object's state, often altered "from the outside" by other objects, is called the **extrinsic state.**



Flyweight - Solution

- *ShapeFactory* has a *HashMap* of *Circle* having key as color (intrinsic state) of the *Circle* object. Whenever a request comes to create a circle of particular color to *ShapeFactory*, it checks the circle object in its *HashMap*, if object of *Circle* found, that object is returned otherwise a new object is created, stored in hashmap for future use, and returned to client.

- *FlyWeightPatternDemo*, our demo class, will use *ShapeFactory* to get a *Shape*object. It will pass information (*red / green / blue/ white/ black/*) to *ShapeFactory* to get the circle of desired color it needs.



Java translation

Create a factory to generate object of concrete class based on given information.

```java
import java.util.HashMap;

public class ShapeFactory {

   private static final HashMap circleMap = new HashMap();

   public static Shape getCircle(String color) {
      Circle circle = (Circle)circleMap.get(color);

      if(circle == null) {
         circle = new Circle(color);
         circleMap.put(color, circle);
         System.out.println("Creating circle of color : " + color);
      }
      return circle;
   }
}
```

ShapeFactory.java

Java translation

Use the factory to get object of concrete class by passing an information such as color.

```
public class FlyweightPatternDemo {
   private static final String colors[] = { "Red", "Green", "Blue", "White", "Black'};
   public static void main(String[] args) {

      for(int i=0; i < 20; ++i) {
         Circle circle = (Circle)ShapeFactory.getCircle(getRandomColor());
         circle.setX(getRandomX());
         circle.setY(getRandomY());
         circle.setRadius(100);
         circle.draw();
      }
   }
   private static String getRandomColor() {
      return colors[(int)(Math.random()*colors.length)];
   }
   private static int getRandomX() {
      return (int)(Math.random()*100 );
   }
   private static int getRandomY() {
      return (int)(Math.random()*100);
   }
}
```

FlyweightPatternDemo.java

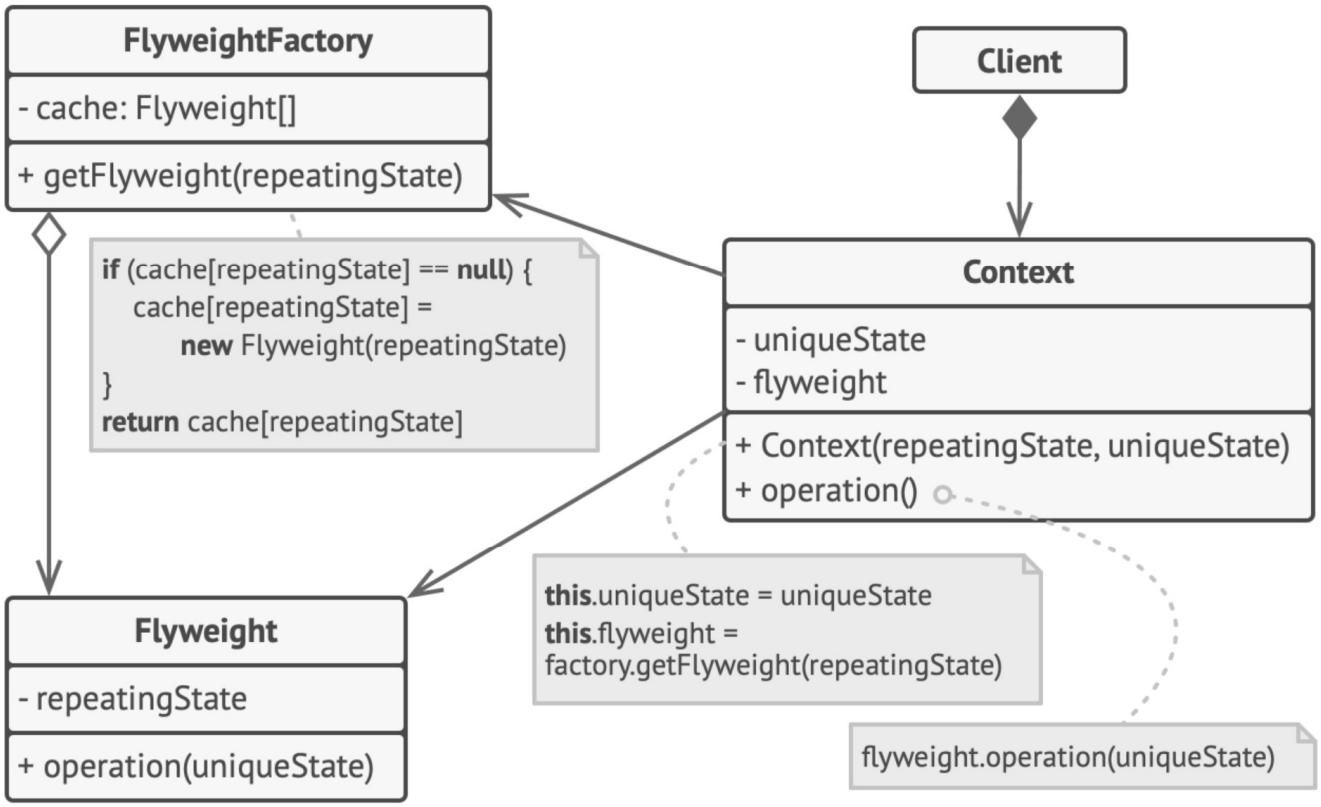
Java translation

- **Verify the output**

```
Creating circle of color : Black
Circle: Draw() [Color : Black, x : 36, y :71, radius :100
Creating circle of color : Green
Circle: Draw() [Color : Green, x : 27, y :27, radius :100
Creating circle of color : White
Circle: Draw() [Color : White, x : 64, y :10, radius :100
Creating circle of color : Red
Circle: Draw() [Color : Red, x : 15, y :44, radius :100
Circle: Draw() [Color : Green, x : 19, y :10, radius :100
Circle: Draw() [Color : Green, x : 94, y :32, radius :100
Circle: Draw() [Color : White, x : 69, y :98, radius :100
Creating circle of color : Blue
Circle: Draw() [Color : Blue, x : 13, y :4, radius :100
Circle: Draw() [Color : Green, x : 21, y :21, radius :100
Circle: Draw() [Color : Blue, x : 55, y :86, radius :100
Circle: Draw() [Color : White, x : 90, y :70, radius :100
Circle: Draw() [Color : Green, x : 78, y :3, radius :100
Circle: Draw() [Color : Green, x : 64, y :89, radius :100
Circle: Draw() [Color : Blue, x : 3, y :91, radius :100
Circle: Draw() [Color : Blue, x : 62, y :82, radius :100
Circle: Draw() [Color : Green, x : 97, y :61, radius :100
Circle: Draw() [Color : Green, x : 86, y :12, radius :100
Circle: Draw() [Color : Green, x : 38, y :93, radius :100
Circle: Draw() [Color : Red, x : 76, y :82, radius :100
Circle: Draw() [Color : Blue, x : 95, y :82, radius :100
```

Flyweight Class diagram

The **Flyweight Factory** manages a pool of existing flyweights. With the factory, clients don't create flyweights directly. Instead, they call the factory, passing it bits of the intrinsic state of the desired flyweight. The factory looks over previously created flyweights and either returns an existing one that matches search criteria or creates a new one if nothing is found.



The **Flyweight** class contains the portion of the original object's state that can be shared between multiple objects. The same flyweight object can be used in many different contexts. The state stored inside a flyweight is called *intrinsic.* The state passed to the flyweight's methods is called *extrinsic.*

Plan

- Flyweight
- Bridge
- Composite
- Proxy
- Adapter

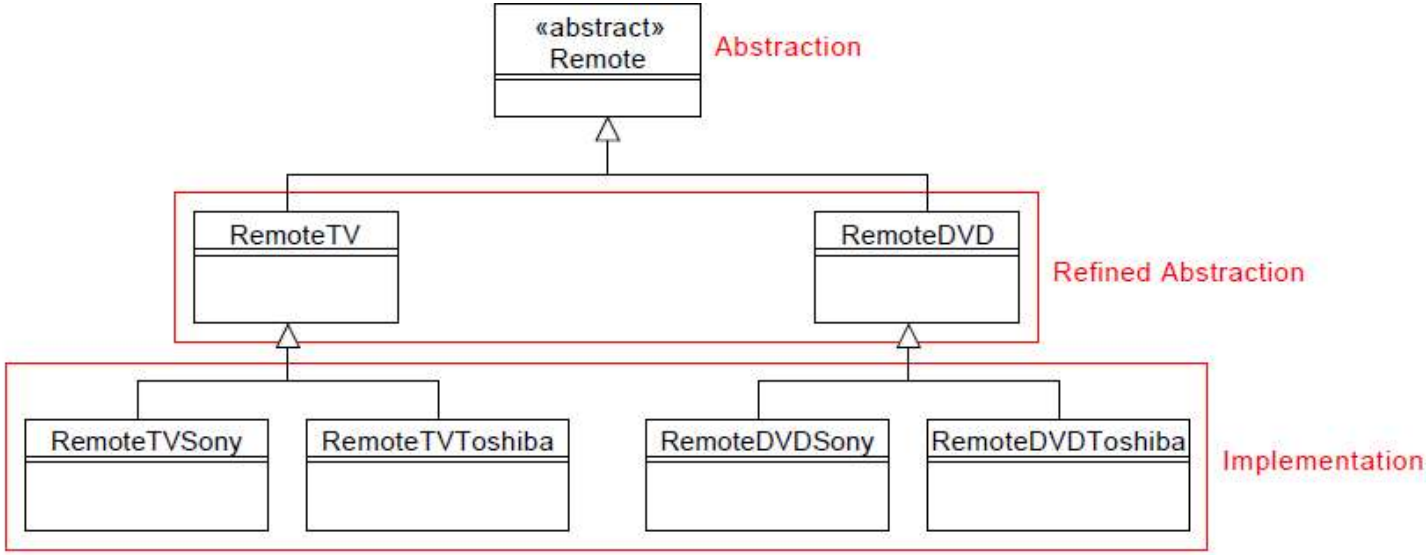
Bridge pattern

Intent

- Decouple an abstraction from its implementation so that the two can vary independently

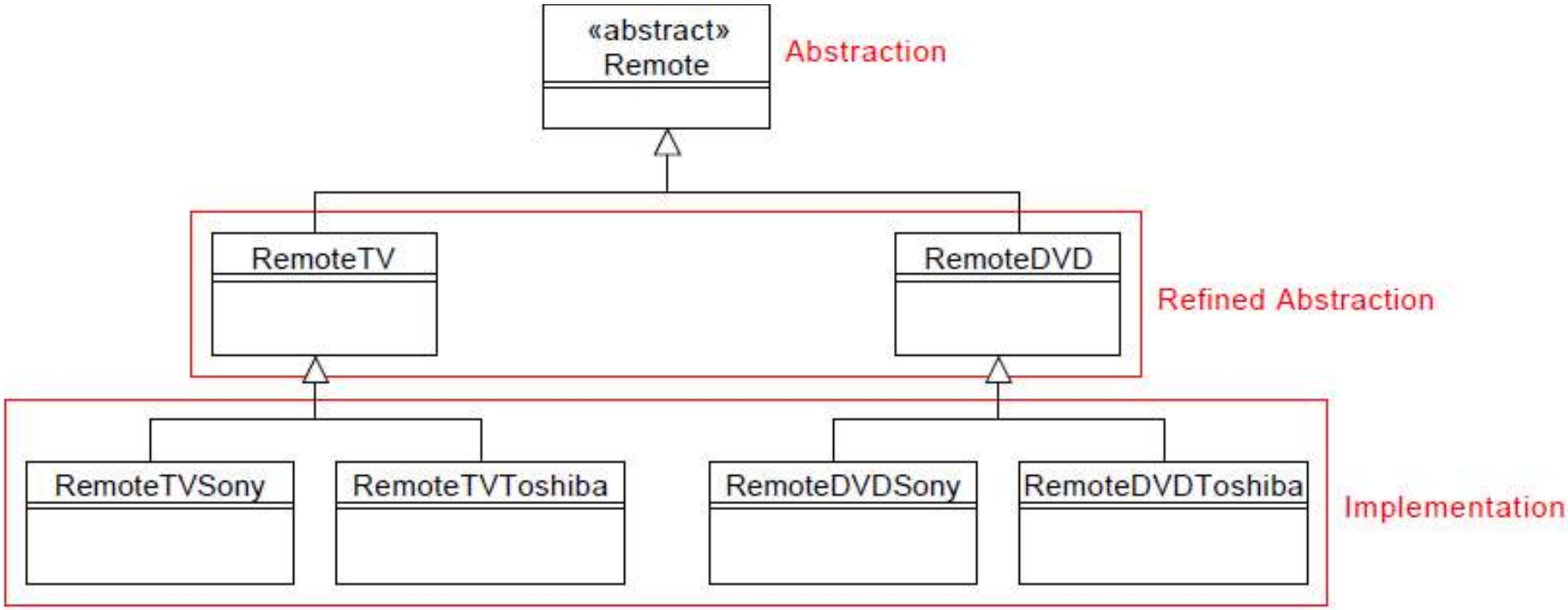
- In this pattern, the abstract class is separated from the implementation class and we provide a bridge interface between them. This interface helps us to make concrete class functionalities independent from the interface implementer class. We can modify these different kind of classes structurally without affecting each other.

Example : Before Bridge Design Pattern



- Remote is an abstract class. It contains:
  - an instance variable sound
  - two methods soundUp and soundDown
  - two abstract methods onPressMiddle and onPressNine
- If it is remote for TV: onPressNine ➔ go to the channel nine.
- If it is remote for DVD: onPressNine ➔pause.
- If it is Sony remote: onPressMiddle ➔open menu.
- If it is Toshiba remote: onPressMiddle ➔turn off.

Bridge pattern problem



- Problems
  - Create a remote for LCD creation of three new data types (RemoteLCD, RemoteLCDSony, RemoteLCDToshiba).
  - Code redundancy (duplication of methods)!

Java Translation

```
public abstract class Remote {
    protected int sound = 0;

    public void soundUp() {
        if(sound < 100)
            sound++;
    }

    public void soundDown() {
        if(sound > 0)
            sound--;
    }
    public abstract void onPressNine();
    public abstract void onPressMiddle();
}
```

Java Translation

```java
public abstract class RemoteDVD extends Remote {
    public void onPressNine() {
        System.out.println("Pause");
    }
}
```

```java
public class RemoteDVDSony extends RemoteDVD {
    public void onPressMiddle() {
        System.out.println("Open Menu");
    }
}
```

```java
public class RemoteDVDToshiba extends RemoteDVD {
    public void onPressMiddle() {
        System.out.println("Turn Off");
    }
}
```

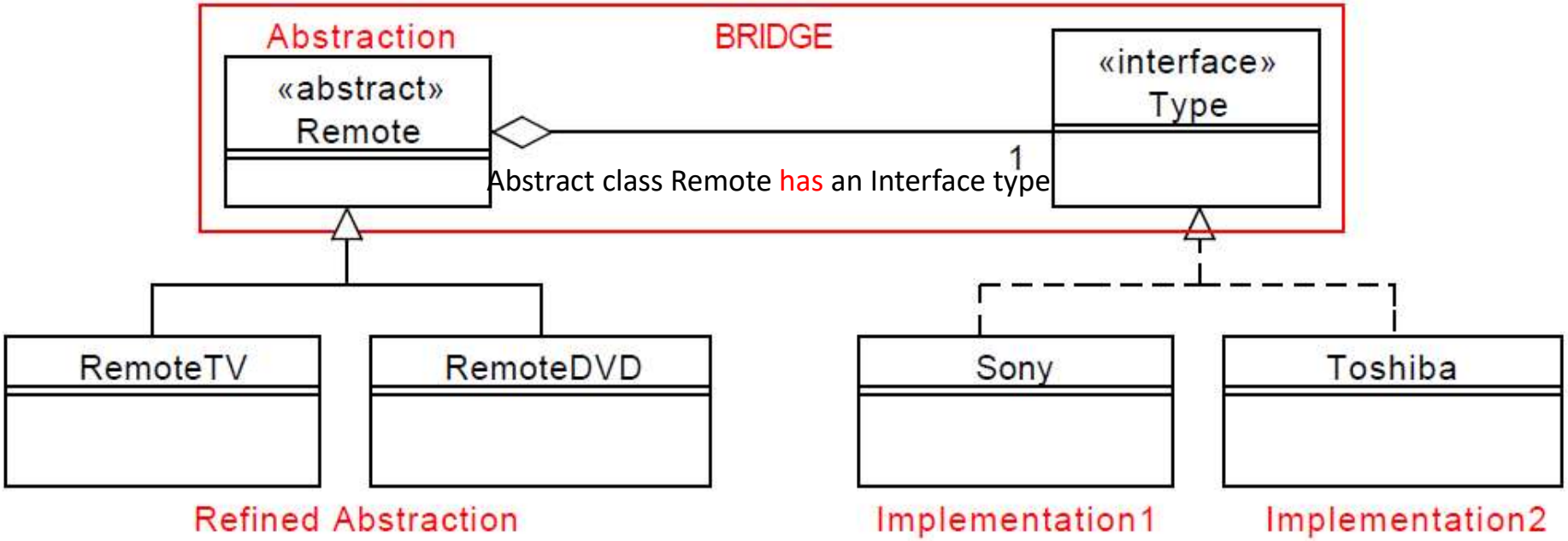
Java Translation

```java
public abstract class RemoteTV extends Remote {
    public void onPressNine() {
        System.out.println("Go To Channel Nine");
    }
}
```

```java
public class RemoteTVSony extends RemoteTV {
    public void onPressMiddle() {
        System.out.println("Open Menu");
    }
}
```

```java
public class RemoteTVToshiba extends RemoteTV {
    public void onPressMiddle() {
        System.out.println("Turn Off");
    }
}
```

Bridge-Solution



The Bridge pattern attempts to solve this problem by switching from inheritance to the object composition. What this means is that you extract one of the dimensions into a separate class hierarchy, so that the original classes will reference an object of the new hierarchy, instead of having all of its state and behaviors within one class.

Java Translation

```java
public interface Type {
    public void onPressMiddle();
}
```

```java
public class Toshiba implements Type {
    @Override
    public void onPressMiddle() {
        System.out.println("Turn Off");
    }
}
```

```java
public class Sony implements Type {
    @Override
    public void onPressMiddle() {
        System.out.println("Open Menu");
    }
}
```

Java Translation

```java
public abstract class Remote {

    public Type type;

    public Remote(Type t) {
        type = t;
    }

    protected int sound = 0;

    public void soundUp() {
        if(sound < 100)
            sound++;
    }

    public void soundDown() {
        if(sound > 0)
            sound--;
    }

    public void onPressMiddle() {
        type.onPressMiddle();
    }
    public abstract void onPressNine();
}
```

Java Translation

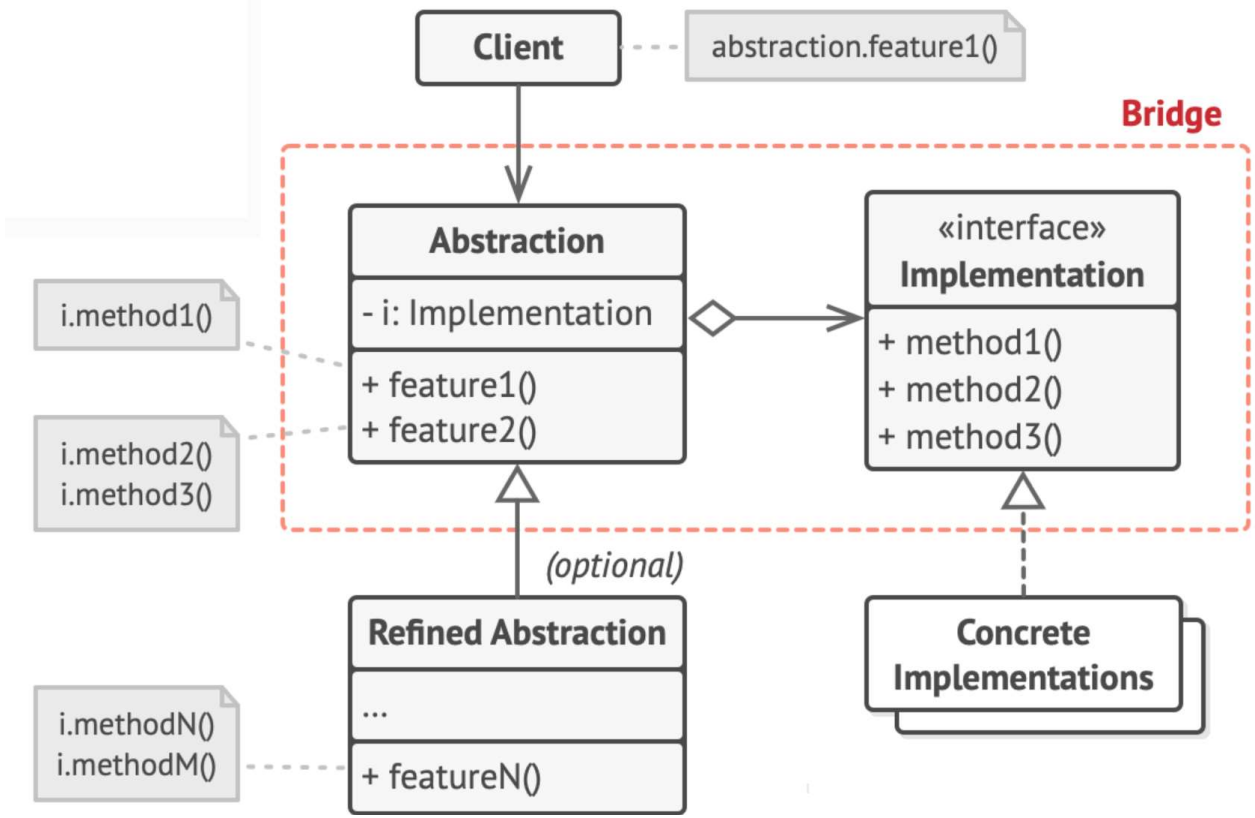
```java
public class RemoteDVD extends Remote {
    public RemoteDVD(Type t) {
        super(t);
    }
    public void onPressNine() {
        System.out.println("Pause");
    }
}
```

```java
public class RemoteTV extends Remote {
    public RemoteTV(Type t) {
        super(t);
    }
    public void onPressNine() {
        System.out.println("Go To Channel Nine");
    }
}
```

Java Translation

```java
public class Test {
    public static void main(String[] args) {
        Remote r1 = new RemoteDVD(new Toshiba());
        Remote r2 = new RemoteTV(new Sony());

        r1.onPressMiddle();
        r2.onPressNine();
    }
}
```

Bridge Pattern UML diagram



Plan

- Flyweight
- Bridge
- Composite
- Proxy
- Adapter

Composite design pattern

Intent

- **Composite** is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

- In other words, when we need to create a structure in a way that the objects in the structure has to be treated the same way, we can apply composite design pattern

- In object-oriented programming, we make a composite object when we have many objects with common functionalities. This relationship is also termed a “has-a” relationship among objects.

Example

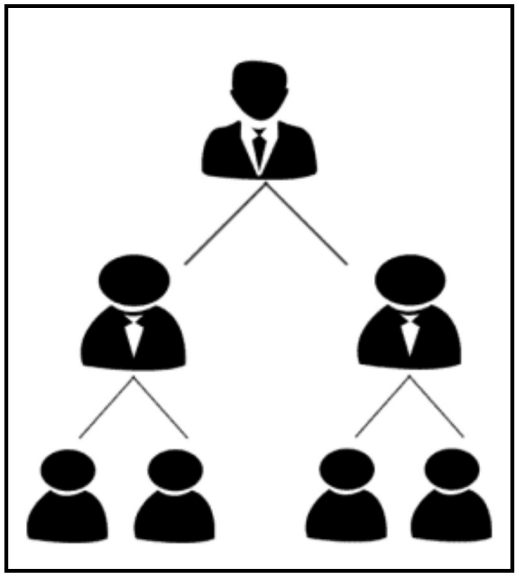
Let's imagine a simple graphic application where users can draw shapes such as circles, squares, and groups of shapes. Each shape can be manipulated individually (moved, resized, colored) or as a group.

- **Leaf**: Individual shapes like a circle or a square.
- **Composite**: A group of shapes that can contain circles, squares, or other groups.
- **Component**: The common interface that defines operations like move, resize, and color.

Users of the application can select a single shape or a group of shapes and apply the same operations. This approach simplifies the management of the application's graphic objects, making operations on complex shapes as easy as on simple shapes.

Composite-Example

We can think of any organization that has many departments, and in turn each department has many employees to serve.



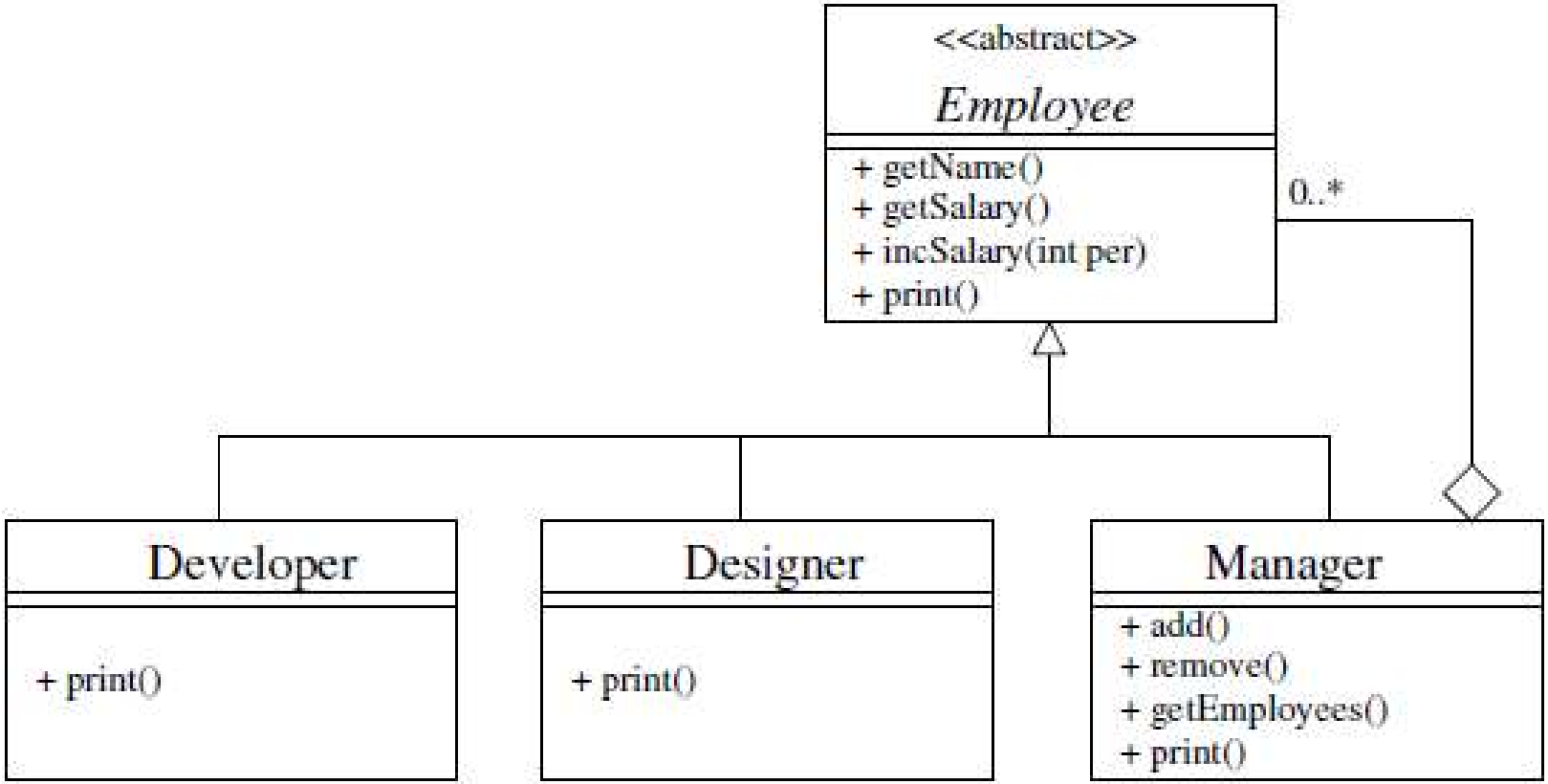
Please note that actually all are employees of the organization.

Groupings of employees create a department, and those departments ultimately can be grouped together to build the whole organization.

Example (Employees)

- You have a set of employees
- An employee could be a manager (of a set of employees), a developer, or a designer

UML Diagram


<<abstract>>
Employee
+ getName()
+ getSalary()
+ incSalary(int per)
+ print()
0..*
Developer
+ print()
Designer
+ print()
Manager
+ add()
+ remove()
+ getEmployees()
+ print()

Java Translation

```java
public abstract class Employee {
   protected String name;
   protected int salary;

   public Employee(String name, int salary) {
      this.name = name;
      this.salary = salary;
   }

   public String getName() {
      return name;
   }

   public int getSalary() {
      return salary;
   }

   public void increaseSalary(int percentage) {
      salary *= 1 + percentage/100.0;
   }

   public abstract void print();
}
```

Java Translation

```java
public class Manager extends Employee {
   protected List<Employee> employees = new LinkedList<Employee>();

   public Manager(String name, int salary) {
      super(name, salary);
   }

   public void add(Employee employee) {
      employees.add(employee);
   }

   public Iterator<Employee> getEmployees() {
      return employees.iterator();
   }

   public void remove(Employee employee) {
      employees.remove(employee);
   }

   public void print() {
      System.out.println("Manager " + name);
   }
}
```

Java Translation

```java
public class Developer extends Employee {
    public Developer(String name, int salary) {
        super(name, salary);
    }

    public void print() {
        System.out.println("Developer " + name);
    }
}
```

```java
public class Designer extends Employee {
    public Designer(String name, int salary) {
        super(name, salary);
    }

    public void print() {
        System.out.println("Designer " + name);
    }
}
```

Java Translation

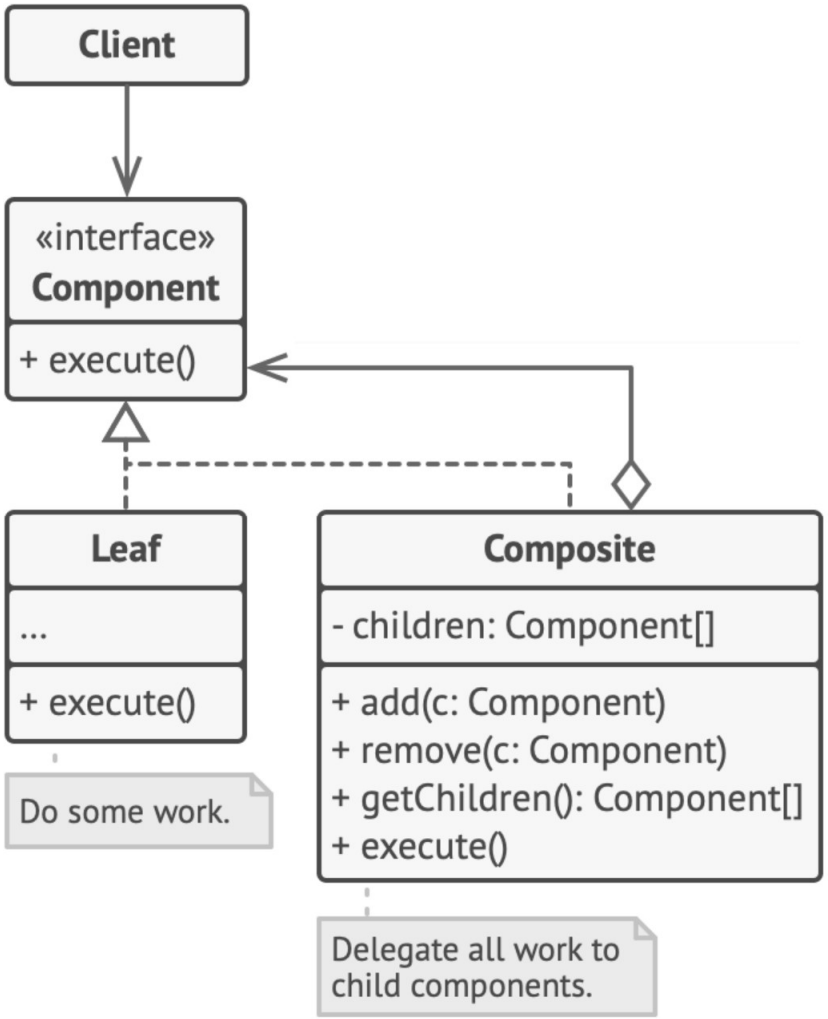
```
public class Company
{
    public static void main (String[] args)
    {
        Developer dev1 = new Developer("Lokesh Sharma", 100 )
        Developer dev2 = new Developer("Vinay Sharma",101 )
        Manager engDirectory = new Manager("Vikram Sharma ", 201)
        engDirectory.add(dev1);
        engDirectory.add(dev2);

        Manager accDirectory = new Manager("Kushagra Garg",200)

        Designer d1 = new Designer  ("Lok rma", 100 )
        Designer d2 = new Designer  ("Vin Sha",101 )
        accDirectory.add(d1);
        accDirectory.add(d2);

        Manager directory = new Manager("ram Vikrma ", 221)
        directory.add(engDirectory);
        directory.add(accDirectory);

    }
}
```

Composite pattern-Solution

The **Component** interface describes operations that are common to both simple and complex elements of the tree.

The **Leaf** is a basic element of a tree that doesn't have sub-elements.

The **Container** (aka *composite*) is an element that has sub-elements: leaves or other containers. A container doesn't know the concrete classes of its children. It works with all sub-elements only via the component interface.
Upon receiving a request, a container delegates the work to its sub-elements, processes intermediate results and then returns the final result to the client.



Plan

- Flyweight
- Bridge
- Composite
- Proxy
- Adapter

Proxy design pattern

Intent

- **Proxy** is a structural design pattern that lets you provide a substitute or placeholder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.

Types of proxies:

- **Protection proxy**: controls access to the original object
- **Virtual proxy**: create an expensive object if needed only
- .....

Protection Proxy Problem

Example:

A very simple real life scenario is our University internet, which restricts few site access. The proxy first checks the host you are connecting to, if it is not part of restricted site list, then it connects to the real internet. This example is based on Protection proxies.

Java Translation-Solution

```java
public interface Internet
{
    public void connectTo(String serverhost) throws Exception;
}
```

```java
public class RealInternet implements Internet
{
    @Override
    public void connectTo(String serverhost)
    {
        System.out.println("Connecting to "+ serverhost);
    }
}
```

Java Translation-Solution

```java
public class ProxyInternet implements Internet
{
    private Internet internet = new RealInternet();
    private static List<String> bannedSites;

    static
    {
        bannedSites = new ArrayList<String>();
        bannedSites.add("abc.com");
        bannedSites.add("def.com");
        bannedSites.add("ijk.com");
        bannedSites.add("lnm.com");
    }

    @Override
    public void connectTo(String serverhost) throws Exception
    {
        if(bannedSites.contains(serverhost.toLowerCase()))
        {
            throw new Exception("Access Denied");
        }

        internet.connectTo(serverhost);
    }

}
```

Java Translation-Solution

```java
public class Client
{
    public static void main (String[] args)
    {
        Internet internet = new ProxyInternet();
        try
        {
            internet.connectTo("geeksforgeeks.org");
            internet.connectTo("abc.com");
        }
        catch (Exception e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

Result

```
Connecting to geeksforgeeks.org
Access Denied
```

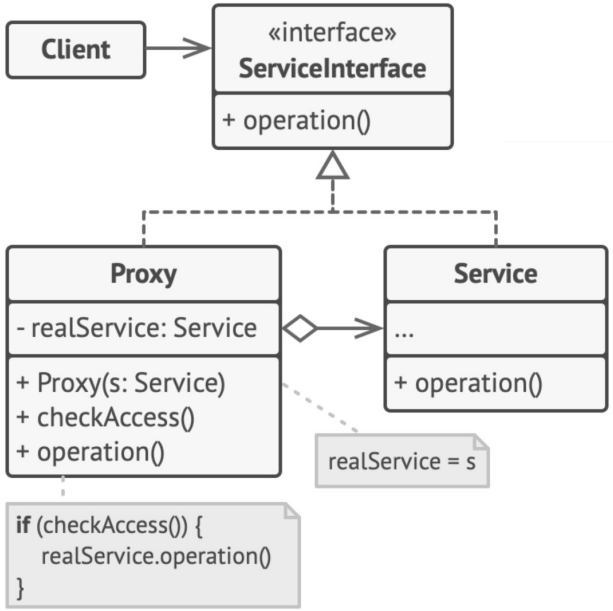
Poxy design pattern Structure

The Proxy pattern suggests that you create a new proxy class with the same interface as an original service object. Then you update your app so that it passes the proxy object to all of the original object's clients. Upon receiving a request from a client, the proxy creates a real service object and delegates all the work to it.

The **Service Interface** declares the interface of the Service. The proxy must follow this interface to be able to mask itself as a service object.

The **Proxy** class has a reference field that points to a service object. After the proxy finishes its processing (e.g., lazy initialization, logging, access control, caching, etc.), it passes the request to the service object.

The **Service** is a class that provides some useful business logic.

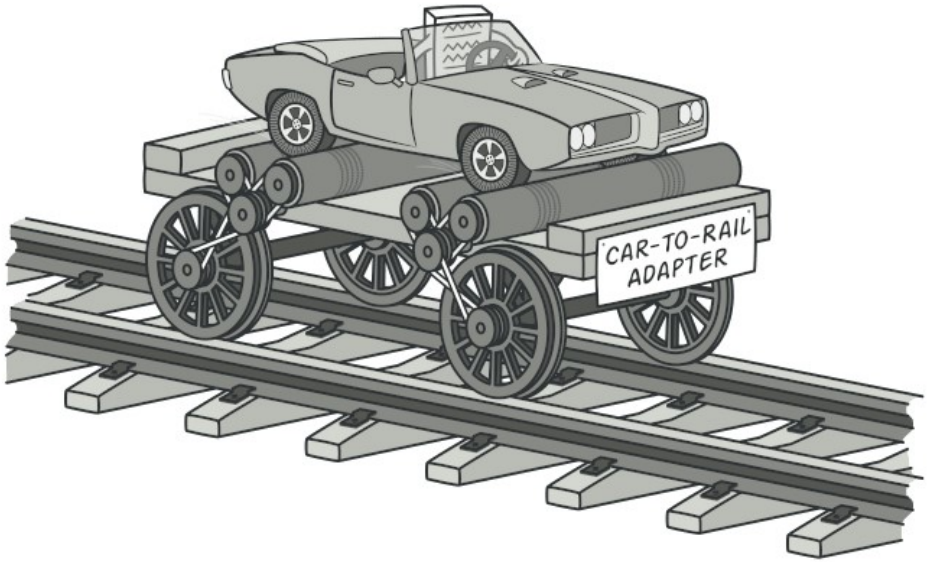


Plan

- Flyweight
- Bridge
- Composite
- Proxy
- Adapter

Adapter Pattern

Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.

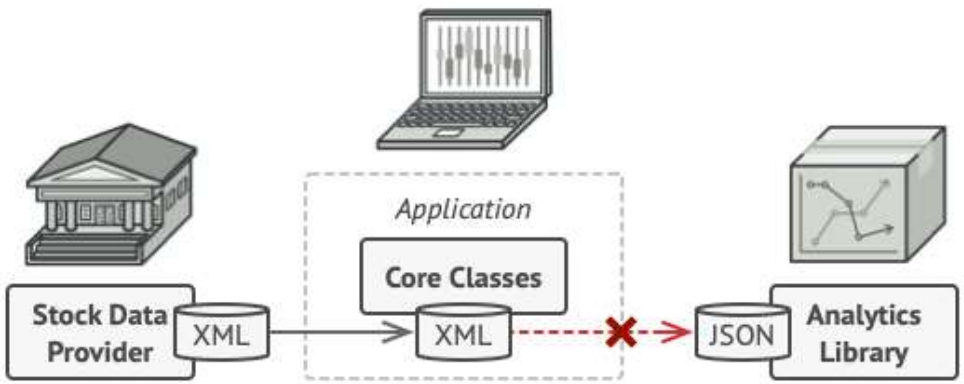


A real life example could be a case of card reader which acts as an adapter between memory card and a laptop. You plugin the memory card into card reader and card reader into the laptop so that memory card can be read via laptop.

Problem

Imagine that you're creating a stock market monitoring app. The app downloads the stock data from multiple sources in XML format and then displays nice-looking charts and diagrams for the user.

- At some point, you decide to improve the app by integrating a smart 3rd-party analytics library. But there's a catch: the analytics library only works with data in JSON format.



You can't use the analytics library "as is" because it expects the data in a format that's incompatible with your app.

Bad Solution

You could change the library to work with XML. However, this might break some existing code that relies on the library. And worse, you might not have access to the library's source code in the first place, making this approach impossible.

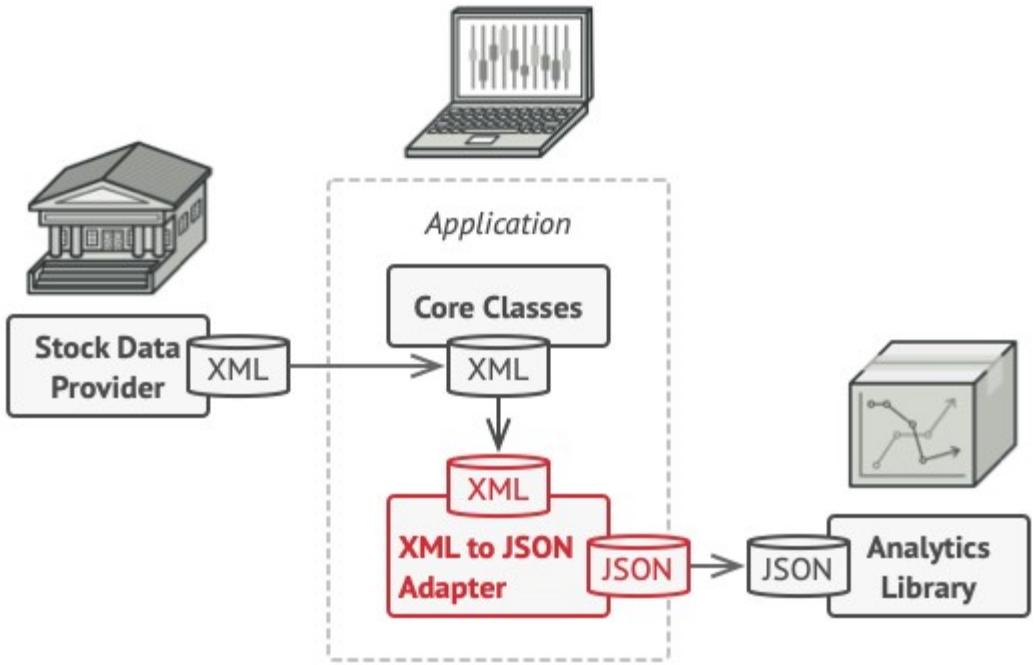
Good Solution

You can create an *adapter*. This is a special object that converts the interface of one object so that another object can understand it.

Here's how it works:

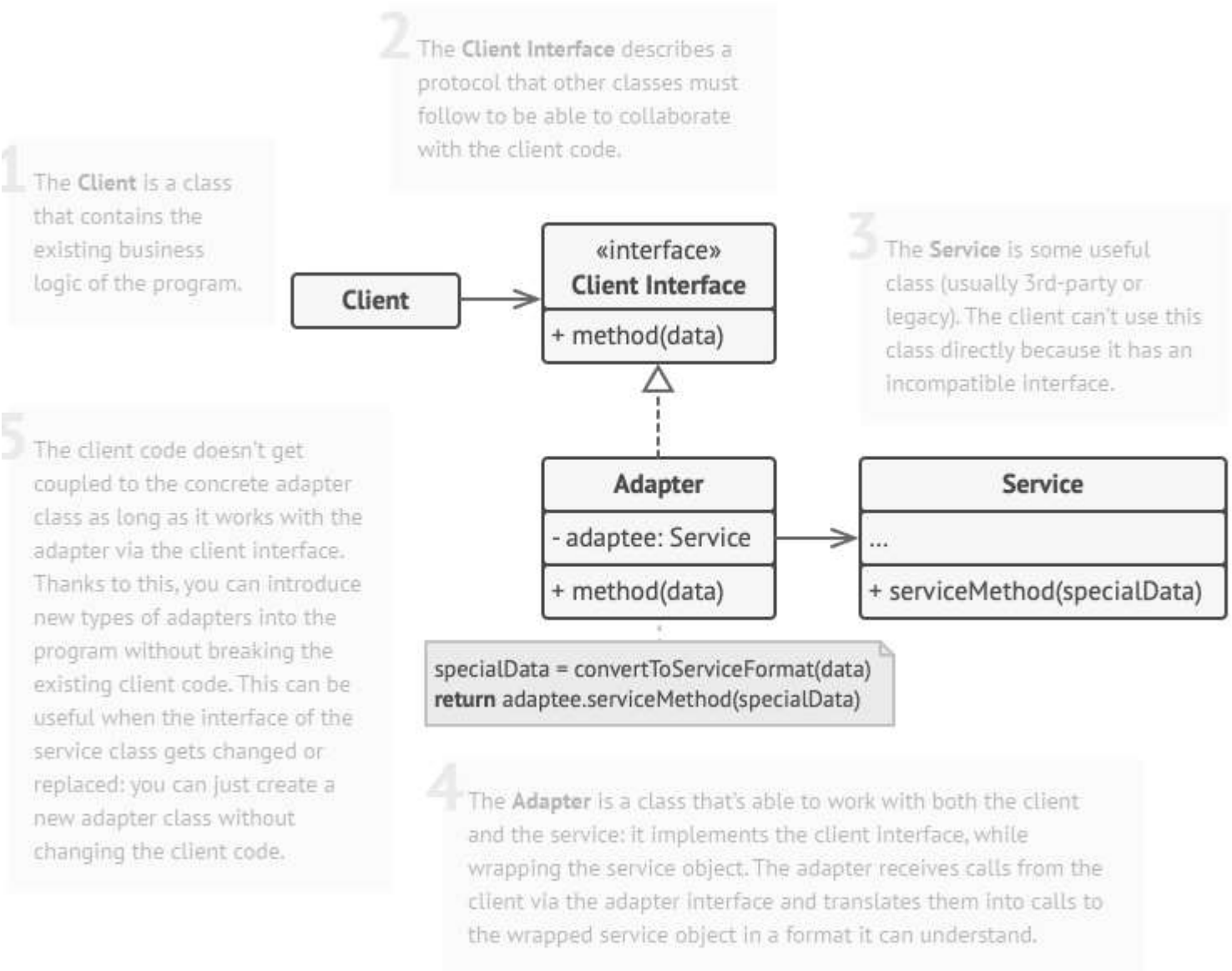
- The adapter gets an interface, compatible with one of the existing objects.
- Using this interface, the existing object can safely call the adapter's methods.
- Upon receiving a call, the adapter passes the request to the second object, but in a format and order that the second object expects.

Good Solution



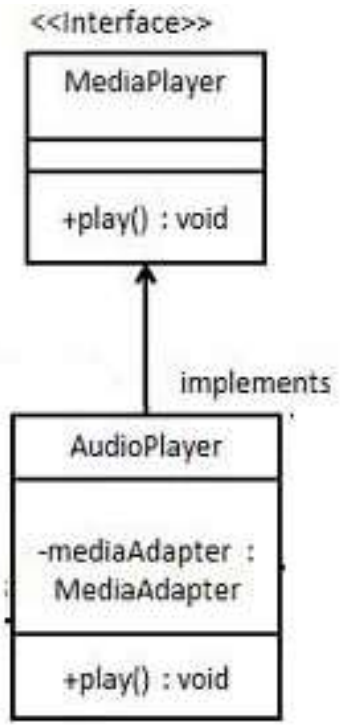
To solve the dilemma of incompatible formats, you can create XML-to-JSON adapters for every class of the analytics library that your code works with directly. Then you adjust your code to communicate with the library only via these adapters. When an adapter receives a call, it translates the incoming XML data into a JSON structure and passes the call to the appropriate methods of a wrapped analytics object.

Structure


2 The Client Interface describes a protocol that other classes must follow to be able to collaborate with the client code.
1 The Client is a class that contains the existing business logic of the program.
Client
«interface»
Client Interface
+ method(data)
3 The Service is some useful class (usually 3rd-party or legacy). The client can't use this class directly because it has an incompatible interface.
5 The client code doesn't get coupled to the concrete adapter class as long as it works with the adapter via the client interface. Thanks to this, you can introduce new types of adapters into the program without breaking the existing client code. This can be useful when the interface of the service class gets changed or replaced: you can just create a new adapter class without changing the client code.
Adapter
- adaptee: Service
+ method(data)
Service
...
+ serviceMethod(specialData)
specialData = convertToServiceFormat(data)
return adaptee.serviceMethod(specialData)
4 The Adapter is a class that's able to work with both the client and the service: it implements the client interface, while wrapping the service object. The adapter receives calls from the client via the adapter interface and translates them into calls to the wrapped service object in a format it can understand.

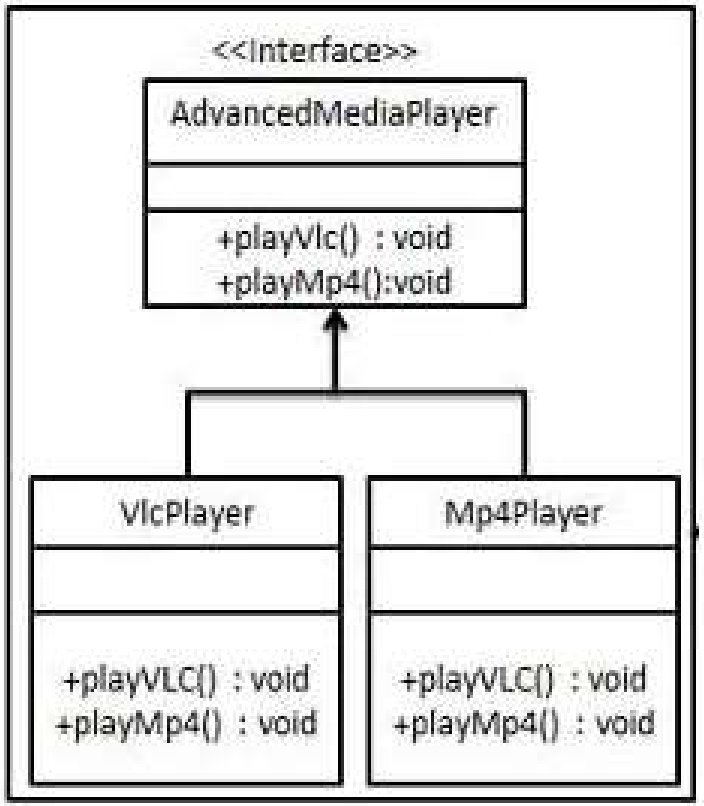
Example

We have a *MediaPlayer* interface and a concrete class *AudioPlayer* implementing the *MediaPlayer* interface. *AudioPlayer* can play mp3 format audio files by default



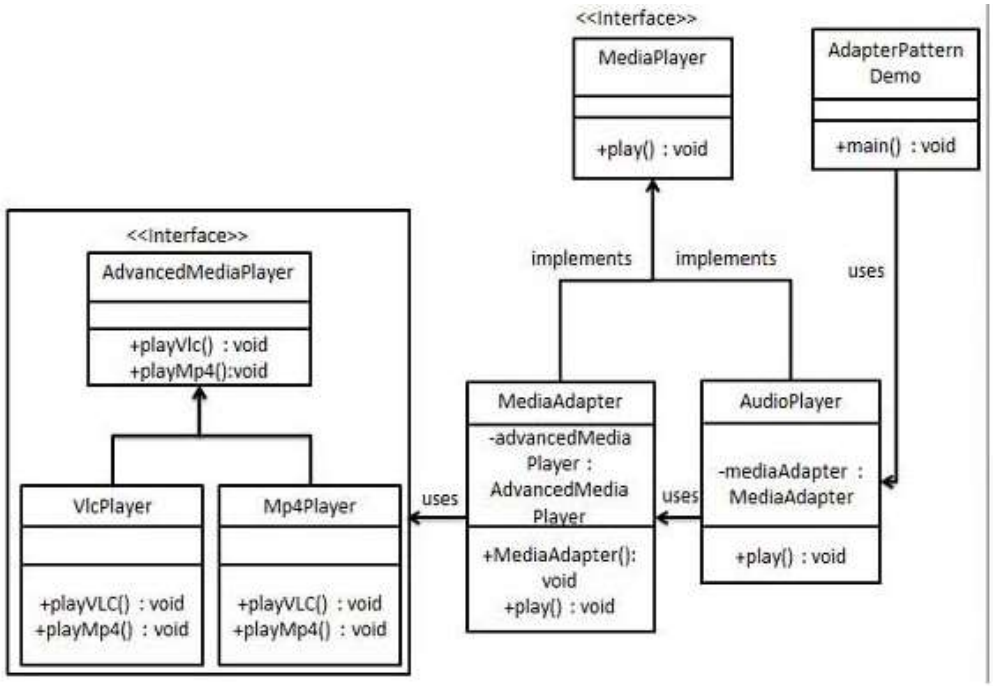
Example

We are having another interface AdvancedMediaPlayer and concrete classes implementing the AdvancedMediaPlayer interface. These classes can play Vlc and Mp4 format files.



Example

We want to make AudioPlayer to play other formats as well. To achieve this, we have created an adapter class MediaAdapter which implements the MediaPlayer interface and uses AdvancedMediaPlayer objects to play the required format.



AudioPlayer uses the adapter class MediaAdapter passing it the desired audio type without knowing the actual class which can play the desired format.

AdapterPatternDemo, our demo class will use AudioPlayer class to play various formats.

Step 1

- Create interfaces for Media Player and Advanced Media Player.

```
public interface MediaPlayer {
   public void play(String audioType, String fileName);
}
```

```
public interface AdvancedMediaPlayer {
   public void playVlc(String fileName);
   public void playMp4(String fileName);
}
```

Step 2

- Create concrete classes implementing the *AdvancedMediaPlayer* interface.

```java
public class VlcPlayer implements AdvancedMediaPlayer{
   @Override
   public void playVlc(String fileName) {
      System.out.println("Playing vlc file. Name: "+ fileName);
   }

   @Override
   public void playMp4(String fileName) {
      //do nothing
   }
}
```

```java
public class Mp4Player implements AdvancedMediaPlayer{

   @Override
   public void playVlc(String fileName) {
      //do nothing
   }

   @Override
   public void playMp4(String fileName) {
      System.out.println("Playing mp4 file. Name: "+ fileName);
   }
}
```

Step 3

- Create adapter class implementing the *MediaPlayer* interface.

```
public class MediaAdapter implements MediaPlayer {

   AdvancedMediaPlayer advancedMusicPlayer;

   public MediaAdapter(String audioType){

      if(audioType.equalsIgnoreCase("vlc") ){
         advancedMusicPlayer = new VlcPlayer();

      }else if (audioType.equalsIgnoreCase("mp4")){
         advancedMusicPlayer = new Mp4Player();
      }
   }

   @Override
   public void play(String audioType, String fileName) {

      if(audioType.equalsIgnoreCase("vlc")){
         advancedMusicPlayer.playVlc(fileName);
      }
      else if(audioType.equalsIgnoreCase("mp4")){
         advancedMusicPlayer.playMp4(fileName);
      }
   }
}
```

Step 4

- Create concrete class implementing the *MediaPlayer* interface.

```
public class AudioPlayer implements MediaPlayer {
   MediaAdapter mediaAdapter;

   @Override
   public void play(String audioType, String fileName) {

      //inbuilt support to play mp3 music files
      if(audioType.equalsIgnoreCase("mp3")){
         System.out.println("Playing mp3 file. Name: " + fileName);
      }

      //mediaAdapter is providing support to play other file formats
      else if(audioType.equalsIgnoreCase("vlc") || audioType.equalsIgnoreCase("mp4")){
         mediaAdapter = new MediaAdapter(audioType);
         mediaAdapter.play(audioType, fileName);
      }

      else{
         System.out.println("Invalid media. " + audioType + " format not supported");
      }
   }
}
```

Step 5

- Use the AudioPlayer to play different types of audio formats.

```
public class AdapterPatternDemo {
   public static void main(String[] args) {
      AudioPlayer audioPlayer = new AudioPlayer();

      audioPlayer.play("mp3", "beyond the horizon.mp3");
      audioPlayer.play("mp4", "alone.mp4");
      audioPlayer.play("vlc", "far far away.vlc");
      audioPlayer.play("avi", "mind me.avi");
   }
}
```

Step 6

- Verify the output.

```
Playing mp3 file. Name: beyond the horizon.mp3
Playing mp4 file. Name: alone.mp4
Playing vlc file. Name: far far away.vlc
Invalid media. avi format not supported
```

Questions?